

What if a bad release could undo itself... in under a minute?

I built a system that releases new software to a few users first, **measures** if it's healthy, and automatically rolls it back if it's not.



Bad release hits
everyone



Bad release hits **a few users**, then disappears

SLO-Driven Progressive Delivery with Automated Rollback ·
Kubernetes · Argo Rollouts · Prometheus

THE PROBLEM

Most teams release like this

- 1 New version replaces the old one, for everyone**

All 5 servers switch at once.



- 2 A hidden bug reaches 100% of users**

The deployment said "success". Users disagree.

- 3 Someone gets an alarm at 3 AM**

...and has to decide, half asleep: "is it bad enough to undo?"

- 4 Rollback is manual, slow and stressful**

Every minute of delay = more angry customers.

THE IDEA, LIKE A RESTAURANT 🍴

Test the new dish on one table first



1 of 5 servers gets the new version, so only
~20% of users see it.

Every 30 seconds a computer checks two numbers for the **new version only**:

❌ How many requests **fail**? ⌚ How **slow** is it?

Healthy?

20% → 40% →
60% → 80% →
100%

Unhealthy?

Removed. Everyone
back on the old
version.

WHY & WHEN TO USE IT

Use this when a bad release costs you

- ✓ You release often: weekly, daily, many times a day
- ✓ Real customers use the service: shops, payments, apps
- ✓ Someone is on call and you want fewer 3 AM decisions
- ✓ You want "is it healthy?" to be a **number**, not an opinion
- ✓ You run on Kubernetes and already have (or want) metrics

Probably not needed when...

- ✗ a tiny internal tool has almost no traffic to measure
- ✗ you release twice a year and can test everything by hand

HOW A TEAM USES IT

Release as usual. The system decides.

1

Agree on a target (SLO)

"99.5% of requests must succeed, 95% must be faster than 300 ms."

2

The app reports its numbers

Requests, errors and speed, collected by Prometheus every 5 seconds.

3

Ship a new version with one command

```
helm upgrade ... --set image.tag=1.2.0
```

4

Argo Rollouts releases it step by step

Checking the numbers for the new version at every step.

5

Watch it live on the dashboard

Promoted  or rolled back . Nobody had to wake up.

ARCHITECTURE

How the pieces talk

GitHub Actions: tests every change: unit tests, security scan, and a full rehearsal on a test cluster



Helm: installs the new version with one command



Argo Rollouts: moves traffic 20% → 100% and makes the final call



"Is the new version healthy?"

Prometheus: collects errors & speed from every server, labelled by version



Grafana: shows it all on a live dashboard

Everything runs on **Kubernetes**; I tested it on **kind** (Kubernetes on my laptop, free).

IMPLEMENTATION · THE DECISION RULE

The new version must pass both checks

✖ Failed requests

$\leq 7.2\%$

= 0.5% error budget $\times$ 14.4.
The same line that would wake up the on-call engineer.

🕒 Speed (95% of requests)

$\leq 300\text{ ms}$

Measured on the new version only, never mixed with the old one.

```
# checked every 30 s, for the NEW version only
successCondition: result[0] ≤ 0.0720
failureLimit: 1           # 2 bad checks → undo
consecutiveErrorLimit: 4 # can't measure? → undo
```

🔒 **Safety first:** if monitoring is broken or the new version gets no traffic, the release is **not** promoted. No proof = no promotion.

DOES IT WORK? I TESTED IT

Real results from my test cluster

✓ Healthy version

Promoted

passed 9/9 checks, 0% errors, 24 ms

✗ 25% of requests failing

47 s

to detect & roll back automatically

🐢 600 ms slower

46 s

to detect & roll back automatically

💥 Monitoring switched off

Refused

to promote an unverified release (60 s)

Tested on a local kind cluster and re-run automatically in GitHub Actions on every change. A lab project, not production traffic.

NEW TO DEVOPS? START HERE 

I wrote a free study guide for this project

No prior knowledge needed. For every tool: **what** it is, **why** it's used, **how** it's connected, and commands to try.

Docker

Kubernetes

kind

Helm

SLOs & error budgets

Prometheus

Argo Rollouts

Grafana

GitHub Actions

Trivy

13

chapters, from
zero

8

hands-on labs

25

interview
questions with
answers

 Also as one **42-page PDF** you can download and print.

TRY IT YOURSELF

Run it on your laptop in ~3 minutes

You need: Docker, kind, kubectl, helm (all free)

```
# 1. get the code
git clone github.com/sufyanahmadkamboh/
    sufyan-devops-slo-progressive-delivery
# 2. build everything (cluster + monitoring + app)
make up
# 3. release a good version → watch it get promoted
scripts/release.sh 1.1.0
# 4. release a broken one → watch it roll back
scripts/release.sh 1.2.0 --inject-errors
# 5. open the dashboard (http://localhost:3000)
kubectl -n monitoring port-forward svc/grafana 3000:3000
```

Then follow the 8 labs in the study guide. 🎓

THANKS FOR READING 🙌

Releases that promote themselves when healthy, and undo themselves when not.



Code, docs & study guide

github.com/sufyanahmadkamboh/sufyan-devops-slo-progressive-delivery



My portfolio

sufyanahmadkamboh.github.io

💬 How does your team decide a release is "good enough"?

Tell me in the comments 🙋